

N100 Financial Intelligence Platform

Analyst Guide

Project Overview

The N100 Financial Intelligence Platform is a financial analytics application developed to analyse companies listed in the Nifty 100 index. It brings together financial data, business intelligence and interactive visualisation to provide a single platform for company analysis.

The application automates the complete workflow, beginning with raw Excel datasets and ending with dashboards, reports and REST APIs. Financial information is cleaned, validated and stored in a SQLite database before key performance indicators are calculated. These insights are then presented through a Streamlit dashboard and a FastAPI backend.

The platform allows users to compare companies, analyse financial performance, screen investments using custom filters and generate downloadable reports. By automating repetitive analytical tasks, it provides faster and more consistent financial analysis.

Main Features

- Automated ETL pipeline
- Financial KPI calculations
- Company profile dashboard
- Financial screener
- Peer comparison
- Sector analytics
- Capital allocation analysis
- FastAPI REST API
- Downloadable reports

Technology Stack

Python, Pandas, NumPy, SQLite, FastAPI, Streamlit, Plotly, Pytest and Git were used to develop the project.

Installation Guide

Ensure Python 3.13 or later and Git are installed before setting up the project.

Clone the repository and move into the project directory.

```
git clone <repository-url>
```

```
cd N100_Financial_Intelligence
```

Create and activate a virtual environment.

```
python -m venv venv
```

```
venv\Scripts\activate
```

Install the required packages.

```
pip install -r requirements.txt
```

Database Setup

Load the source datasets into the SQLite database.

```
python src/etl/loader.py
```

Validate the imported data.

```
python src/validation/validator.py
```

Running the Dashboard

```
streamlit run src/dashboard/app.py
```

Running the API

```
uvicorn src.api.main:app --reload --port 8000
```

Swagger documentation is available at:

```
http://127.0.0.1:8000/docs
```

Running Tests

```
pytest
```

```
pytest tests --html=reports/pytest_report.html
```

After completing these steps, the dashboard, API and testing environment are ready for use.

ETL Pipeline

The ETL (Extract, Transform and Load) pipeline prepares all financial data before it is used by the dashboard or API.

Raw Excel datasets containing company information, balance sheets, cash flow statements, market data and financial ratios are imported using Pandas. During processing, duplicate records are removed, company identifiers are validated and inconsistent values are filtered using Data Quality rules.

Once the data has been cleaned, financial KPIs such as Return on Equity, Debt-to-Equity Ratio, Operating Profit Margin and Revenue CAGR are calculated and stored in the SQLite database.

The processed database serves as the single source of truth for the entire application, ensuring that the dashboard, APIs and generated reports all use the same validated information.

ETL Workflow

1. Import raw Excel datasets.
2. Clean and validate the data.
3. Apply Data Quality rules.
4. Calculate financial KPIs.
5. Store processed data in SQLite.
6. Supply validated data to the dashboard, API and reporting modules.

This automated workflow improves consistency, reduces manual effort and provides reliable financial data for analysis.

Page 4 – Streamlit Dashboard

Streamlit Dashboard

The Streamlit dashboard provides a simple and interactive interface for exploring financial information without requiring any knowledge of SQL or programming. It retrieves processed data from the SQLite database and presents it using tables, charts and summary metrics.

The dashboard is organised into several modules, each focusing on a different aspect of financial analysis.

Home

The Home screen provides an overview of the platform and displays key financial indicators, helping users quickly understand the available analytical features.

Company Profile

This screen presents detailed information for an individual company, including business description, financial ratios, valuation metrics and historical performance.

Financial Screener

The screener allows users to filter companies using financial metrics such as Return on Equity, Debt-to-Equity Ratio, Revenue CAGR and Free Cash Flow. The filtered results can be exported for further analysis.

Peer Comparison

This section compares companies within the same peer group using financial metrics and percentile rankings, making it easier to benchmark performance against competitors.

Sector Analysis

The Sector screen summarises the performance of companies within each industry, allowing users to identify trends and compare sectors.

Reports

The Reports screen provides access to generated reports and analytical outputs, including downloadable Excel and CSV files.

Overall, the dashboard provides a central location for analysing financial performance in a clear and user-friendly manner.

Financial Screener

The Financial Screener enables users to identify companies that satisfy specific financial criteria. Rather than manually reviewing each company, users can apply filters to narrow the list of companies based on investment preferences.

The screener supports several financial filters, including:

- Minimum Return on Equity (ROE)
- Maximum Debt-to-Equity Ratio
- Minimum Free Cash Flow
- Sector Selection
- Revenue CAGR
- Profit CAGR
- Maximum Price-to-Earnings Ratio

Multiple filters can be combined to produce more targeted results. For example, a user may search for companies with a high ROE, low debt and positive Free Cash Flow within a particular sector.

The filtered results include company name, sector, financial ratios and composite quality score. Results are automatically ranked according to the calculated composite score, allowing stronger companies to appear first.

Users can also export the screened results as a CSV file for further analysis in Excel or other analytical tools.

The screener provides a fast and flexible method of identifying companies that match predefined investment criteria while significantly reducing manual effort.

Reports and Analytics

The platform generates a variety of analytical reports that summarise financial performance and support further analysis. These reports are created automatically during different stages of the project and are saved within the output directory.

Examples of generated outputs include valuation summaries, portfolio statistics, peer comparison reports, validation reports and cash flow intelligence reports.

Company tearsheets provide a concise summary of an individual company's financial position, including profitability, leverage, cash flow and valuation metrics. Sector reports combine similar information for groups of companies operating within the same industry.

Validation reports identify any data quality issues detected during the ETL process, while audit reports record the status of each imported dataset.

Performance reports generated during testing include API response times, dashboard performance measurements and automated test summaries.

Most reports are available in Excel, CSV or PDF format, allowing users to review the information offline or share it with other stakeholders.

These reports provide a reliable record of the analytical results produced by the platform and support further financial investigation when required.

FastAPI Guide

The project includes a FastAPI backend that provides programmatic access to the processed financial data. The API allows external applications to retrieve company information, financial metrics and analytical results without accessing the database directly.

The available endpoints include:

- Health Check
- Company Information
- Financial Screener
- Sector Analytics
- Peer Comparison
- Market Capitalisation
- Portfolio Statistics
- Company Documents

The API follows REST principles and returns data in JSON format. All endpoints include appropriate HTTP status codes and validation to ensure reliable responses.

The project also includes interactive Swagger documentation, making it easy to explore and test each endpoint through a web browser.

Swagger can be accessed by starting the FastAPI server and opening:

`http://127.0.0.1:8000/docs`

Example request:

`GET /api/v1/companies/TCS`

Example response:

The endpoint returns company information, sector details and the latest financial ratios for Tata Consultancy Services.

The FastAPI backend acts as the communication layer between the database and external applications, enabling seamless integration with the Streamlit dashboard and any future client applications.

Testing

Testing was carried out throughout the development of the N100 Financial Intelligence Platform to ensure that each component behaved as expected. The project includes unit tests, API tests and integration tests covering the ETL pipeline, KPI calculations and FastAPI endpoints.

Unit tests were written using Pytest to verify individual functions such as financial ratio calculations, CAGR calculations, cash flow metrics and data validation rules. These tests confirm that the analytical engine produces accurate and consistent results for a variety of input scenarios.

API tests were developed to verify that each FastAPI endpoint returned the correct HTTP status codes and response data. These tests included company information, financial screener, sector analytics, peer comparison and portfolio statistics.

Integration testing was performed by running the FastAPI backend and Streamlit dashboard simultaneously. This ensured that the dashboard successfully retrieved information from the API and displayed the correct data without errors.

Performance testing was also completed by sending multiple concurrent requests to the API and measuring response times. The application successfully handled all requests within the required performance limits.

The complete test suite can be executed using the following command.

```
pytest
```

An HTML report can also be generated for reviewing the overall test results.

```
pytest tests --html=reports/pytest_report.html
```

Regular testing throughout development helped identify issues early and improved the overall reliability of the application.

Troubleshooting

This section outlines some of the most common issues that may occur while running the project together with their recommended solutions.

Database not found

Ensure that the SQLite database has been created by running the ETL loader before starting the dashboard or API.

Module not found

Activate the project's virtual environment and install all required packages using the requirements file.

FastAPI server not starting

Check whether port 8000 is already being used by another application. If necessary, stop the existing process or use a different port.

Streamlit dashboard not loading

Confirm that the FastAPI server is running correctly and that the SQLite database contains the required tables and data.

Missing reports or output files

Run the appropriate analytics or reporting scripts before attempting to access generated files.

Test failures

Verify that all dependencies are installed, the database is available and the latest source code has been saved before running the test suite.

Following these checks resolves most common issues encountered during development and helps maintain a stable working environment.

Future Improvements

The current platform provides a solid foundation for financial analysis, but several enhancements could further improve its functionality and scalability.

One potential improvement is migrating from SQLite to PostgreSQL to support larger datasets and multiple concurrent users. This would make the application more suitable for production environments.

Containerising the application with Docker would simplify deployment and ensure consistent execution across different operating systems. A Continuous Integration and Continuous Deployment (CI/CD) pipeline could also be introduced to automate testing and deployment.

Adding user authentication and role-based access control would improve security by restricting access to sensitive analytical features and reports.

The platform could also be extended to retrieve live market data from external financial APIs, allowing users to analyse real-time stock prices and valuation metrics.

Future versions may include portfolio management tools, watchlists, price alerts and predictive analytics using machine learning models. Cloud deployment on platforms such as Microsoft Azure or AWS would further improve accessibility and scalability.

These enhancements would transform the project from an academic financial analytics platform into a production-ready solution capable of supporting a wider range of users and business requirements.